Reporte Técnico: Analizador Sintáctico con Bison y Flex

Diseño de Gramática LALR(1) y Generación de Árbol de Análisis con Graphviz

Asignatura: Automatas y Compiladores

Integrantes: Alondra Bautista Flores, Amelie Leocadio Nolasco, José Juan Carlos Verdi Baltazar

Docente / Colaborador: Doctor Ismael Domínguez Jiménez

1. Investigación: ¿Qué hace un Analizador Sintáctico?

El **analizador sintáctico** (o *parser*) es la segunda fase fundamental de un compilador. Su función principal consiste en tomar la secuencia de tokens producida por el analizador léxico (Flex) y verificar si cumple con las reglas formales estructuradas en una **Gramática Libre de Contexto (CFG)**.

- **Validación Gramatical:** Asegura que las expresiones del lenguaje sigan una jerarquía sintáctica válida.
- **Árbol de Análisis Sintáctico (AST):** Representa de forma jerárquica las relaciones de precedencia entre componentes del programa.
- **Manejo de Errores:** Captura desviaciones sintácticas y emite diagnósticos para la recuperación del proceso de compilación.

2. Especificación en Bison (`parser.y`)

```
%{
#include <stdio.h>
#include <stdlib.h>
void yyerror(const char *s);
int yylex(void);
%}

%token NUMBER
%left '+' '-'
%left '*' '/'

%%
input: /* vacío */ | input line ;

line: '\n' | expr '\n' { printf("Resultado = %d\n", $1); } ;

expr:
    NUMBER          { $$ = $1; }
  | expr '+' expr   { $$ = $1 + $3; }
  | expr '-' expr   { $$ = $1 - $3; }
  | expr '*' expr   { $$ = $1 * $3; }
  | expr '/' expr   {
                        if ($3 == 0) { yyerror("División por cero"); YYERROR; }
                        else { $$ = $1 / $3; }
                    }
  | '(' expr ')'     { $$ = $2; }
;
%%

void yyerror(const char *s) { fprintf(stderr, "Error sintáctico: %s\n", s); }
int main(void) { return yyparse(); }
```

3. Especificación Léxica en Flex (`scanner.l`)

```
%{
#include "parser.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t]       ; /* Ignorar espacios */
\n          { return '\n'; }
"+"         { return '+'; }
"-"         { return '-'; }
"*"         { return '*'; }
"/"         { return '/'; }
"("         { return '('; }
")"         { return ')'; }
.           { printf("Carácter no reconocido: %s\n", yytext); }
%%
int yywrap(void) { return 1; }
```

4. Generación del Árbol Sintáctico (Graphviz) e Integración

Para cumplir con el requerimiento extra de graficar el árbol sintáctico generado por Bison:

Paso	Comando de Terminal	Descripción
1. Compilar Bison con Gráfico	`bison -d -g parser.y`	Genera los encabezados y el archivo DOT (`parser.gv`).
2. Generar Imagen PNG	`dot -Tpng parser.gv -o arbol_sintactico.png`	Convierte el grafo sintáctico mediante Graphviz.
3. Enlazar Ejecutable	`flex scanner.l && gcc lex.yy.c parser.tab.c -o analizador`	Enlaza el código C resultante y genera el analizador.